

Hello there.



/CAPPS

/CAPPS

Architecting the First 2s of App Start

What actually happens during the first two
seconds of app startup

Users bounce at 3s and uninstall at 5s

As page load goes from:

- **1s** to **3s** the probability of bounce **increases 32%**
- **1s** to **5s** the probability of bounce **increases 90%**
- **1s** to **10s** the probability of bounce **increases 123%**

Source: Google/SOASTA Research, 2017.

Here is how the first 2s break down in practice

T=0ms

Cold Start

Breaking down the collision of native thread initialization, bytecode execution, and the first UI paint.

T=200ms

Module Evaluation

Optimizing module resolution and keeping the initial bundle lightweight to unblock the thread.

T=800ms

State Orchestration

Centralizing async tasks into a single state machine to safely unlock the router.

T=2000ms

Observability

Using native traces to measure the boot time without impacting JS performance.

At **T=0ms**, the JS engine kicks off execution

- The OS paints the splash screen on the main thread, while a separate background thread boots Hermes and loads your JavaScript bundle
- The UI cannot render until the JS thread finishes evaluating that initial bundle
- If the JS thread is busy doing heavy lifting during this phase, the app stays frozen on the splash screen

At **T=200ms**, module evaluation runs the required imports

-  Code written at the top level of a file executes the exact millisecond the file is resolved
- Bare React Native defaults to **inlineRequires**, deferring imports to their exact execution block

Before:

```
var _heavy = require(_dependencyMap[2]);  
  
function onPress() { _heavy.heavyCompute(); }
```

After:

```
function onPress() { require(_dependencyMap[2]).heavyCompute(); }
```

At **T=200ms**, module evaluation runs the required imports (ctd.)

- In Expo, you will need to override Metro defaults to enable deferred inline requires:

```
config.transformer.getTransformOptions = async () => {  
  return {  
    transform: {  
      experimentalImportSupport: true,  
      inlineRequires: true,  
    },  
  };  
};
```

At **T=800ms**, state orchestration resolves the background tasks

-  We have to resolve multiple parallel calls before we even know which screen to show the user
-  If the router tries to guess and render early, you get a nasty flash of the wrong UI
- Hold the native splash screen until all startup checks clear
- Enforce a hard timeout so slow networks open to a graceful fallback, never a frozen splash

At **T=2000ms**, observability captures the final boot time

-  Measuring boot time inside React misses the module evaluation phase entirely
- Instead, a native Sentry transaction measures the real start time from outside the React tree
- We use custom spans inside that transaction to pinpoint exactly which check is bottlenecking the boot

/CAPPS

Live Demo

Time to find out if this still works when
people are watching

Key takeaways from fixing a 7s splash screen

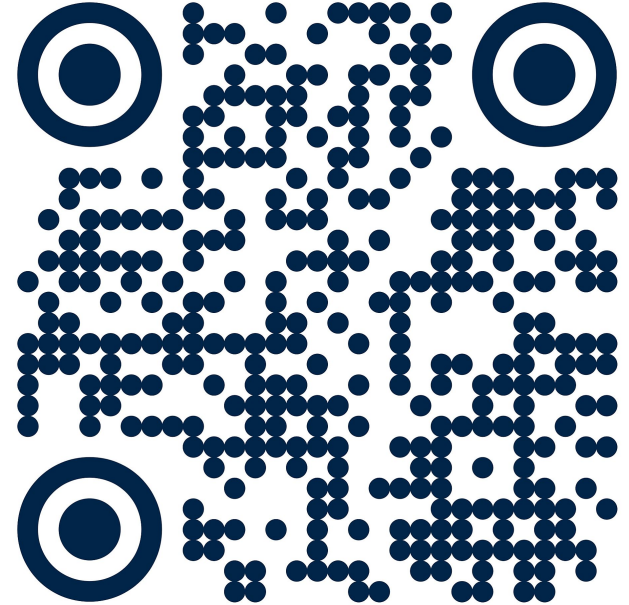
- `inlineRequires` defers module evaluation until access, while `React.lazy` defers component loading until render
- State orchestration keeps the boot architecture clean, predictable, and easy to unit test
- Native observability reveals the exact outliers that cause silent performance hits in Sentry

Questions to take home

- Can we design UI loading patterns that make heavy module evaluation feel instantaneous to the user?
- If a deep link arrives while startup is still resolving, how do you preserve and replay that intent once it completes?
- How do you distinguish between slow code and a user's slow device/network in your telemetry?

Check out the code

github.com/icapps/reactnative_app_start_demo



Thank
you.

Let's work together